

Feladat – MAUI Todo CRUD MVVM alkalmazás**Cél**

Fejlesszen egy .NET MAUI mobilalkalmazást, amely egy lokálisan futó REST API-val kommunikál.

A feladat célja az MVVM architektúrális minta, a Dependency Injection, valamint a RESTful API-hívásokon keresztül megvalósított aszinkron CRUD műveletek gyakorlása.

A feladat során azt gyakorolja, hogyan építhető fel egy modern MAUI alkalmazás úgy, hogy külső adatforrással kommunikál, miközben a felhasználói felület logikája, az adatmodellek és az üzleti logika jól elkülönülnek egymástól.

A feladat áttekintése

Fejlesszen egy kisebb MauiTodoCrudMVVM nevű MAUI mobilalkalmazást, amely kapcsolódik a már meglévő Todo API-hoz, és lehetővé teszi a felhasználó számára:

- a Todo elemek listájának lekérését a REST API-ból,
- új Todo elem hozzáadását,
- meglévő Todo elem szerkesztését,
- Todo elem törlését,
- a Todo elemek szűrését, például:
 - Összes,
 - Aktív,
 - Befejezett.

Minden hálózati műveletet aszinkron módon, az async / await minta használatával kell végrehajtani.

Architektúrális követelmények

Az alkalmazásnak szigorúan követnie kell az MVVM (Model–View–ViewModel) struktúrát.

1. Model réteg

- Definiáljon egy modellosztályt, amely egy Todo entitást reprezentál. Az osztály mezőinek meg kell egyezniük az API JSON struktúrájával.
- A modell egyszerű C# osztály, azaz POCO legyen, felhasználói felülethez kapcsolódó kód nélkül.

2. ViewModel réteg

- Valósítsa meg az adatkötést a View és a Model között.
- Minden ViewModel osztály öröklődjön egy közös bázisosztályból, amely implementálja az INotifyPropertyChanged interfészt.
- A ViewModel tartalmazza:
 - az adatkötéshez szükséges megfigyelhető tulajdonságokat a beviteli mezők és listák adatai számára,
 - a CRUD műveletek logikáját, vagyis az API service-t meghívó metódusokat,
 - a gombokhoz és egyéb felhasználói műveletekhez tartozó Command objektumokat.

3. View réteg – oldalak

- A View réteg XAML oldalakból álljon, amelyek a hozzájuk tartozó ViewModel-ekhez vannak kötve.
- A Todo adatok megjelenítéséhez és szerkesztéséhez használjon data bindingot.
- Minden oldal egy logikailag elkülönülő funkciót valósítson meg:
 - egy oldal a Todo elemek listázásához és szűréséhez,
 - egy oldal Todo elem létrehozásához vagy szerkesztéséhez,
 - opcionálisan egy statisztikai vagy részletező oldal.

4. Service réteg

- Hozzon létre külön service osztályt, például TodoApiService néven, amely a REST API-val történő kommunikációért felel.
- A service valósítsa meg az összes CRUD kérést:
 - GET – összes elem vagy egyetlen elem lekérése,
 - POST – új elem létrehozása,
 - PUT – meglévő elem módosítása,
 - DELETE – elem törlése.
- A hálózati hívások aszinkron módon történjenek, és értelmezhető eredményt vagy hibaüzenetet adjanak vissza.

Technikai megvalósítási irányelvek

1. Dependency Injection (DI)

- Használja a MAUI beépített Dependency Injection rendszerét a service-ek és ViewModel-ek regisztrálására.
- A REST API eléréséhez szükséges HttpClient példányt a MauiProgram.cs fájlban konfigurálja az AddHttpClient() metódus használatával.
- A service osztályt konstruktorinjektálással adja át a ViewModel-eknek.

2. Command minta

- Használjon külön command osztályokat, amelyek egy közös RelayCommandBase osztályból származnak.
- A felhasználói felület minden gombja – például Add, Update, Delete, Refresh – Commandot hajtson végre ahelyett, hogy közvetlenül metódust hívna.
- A commandok kezeljék az engedélyezett és letiltott állapotot is az alkalmazás aktuális állapota alapján, CanExecute logika használatával.

3. Data Binding

- A View és a ViewModel közötti minden interakció data binding segítségével történjen, ne code-behind eseménykezelőkön keresztül.
- A XAML-ben használja az x:DataType attribútumot a típusbiztos binding megvalósításához.
- A ViewModel minden adatváltozás esetén küldjön property-changed értesítést.

4. Aszinkron hálózati kommunikáció

- Minden REST API műveletet aszinkron metódusokkal és await használatával valósítson meg.
- Kezelje a lehetséges kapcsolati hibákat és időtúllépéseket, és jelenítsen meg megfelelő hibaüzeneteket a felhasználói felületen.
- A hálózati kommunikáció közben az alkalmazás felülete maradjon reszponzív.

5. Felhasználói felület

Az alkalmazás fő struktúrája TabbedPage használatával készüljön, és a funkcionalitás több lapra legyen szétválasztva:

- **Todos Page** – megjeleníti a listát, a szűrési lehetőségeket és az alapvető műveleteket,
- **Editor Page** – lehetővé teszi új elemek hozzáadását vagy meglévők szerkesztését,

- **Statistics Page** – opcionálisan megjelenítheti a feladatok számát vagy a teljesítési arányt.

A kialakítás legyen egyszerű és áttekinthető; a hangsúly a funkcionalitáson legyen, ne a dekoráción.

Hálózati beállítások

Az alkalmazás a korábban elkészített, lokálisan futó Todo REST API-hoz kapcsolódik.

- **Windows esetén** az API alapcíme: <http://localhost:5071>
- **Android Emulator esetén** az API alapcíme: <http://10.0.2.2:5071>

Szükséges NuGet csomagok

A projekt elkészítése előtt ellenőrizze, hogy az alábbi csomagok hozzá vannak adva a projekthez:

Csomag	Funkció
Microsoft.Extensions.Http	Az AddHttpClient() használatát biztosítja Dependency Injection mellett
Microsoft.Extensions.Logging.Debug	Az AddDebug() használatát biztosítja fejlesztés közbeni konzolos naplózáshoz
System.Net.Http.Json	Kényelmes JSON szerializációs és deszerializációs segédmetódusokat biztosít
CommunityToolkit.Mvvm – opcionális	Attribútumalapú property- és command-generáláshoz használható